ME-2 Digital Twin Baseline Construction

Week 7: Environmental Compensation

Blayne Montaque

Spring 2026

1 Purpose

The purpose of this stage is to incorporate environmental compensation into the behavioral digital twin so that normal temperature-induced sensor drift is not incorrectly interpreted as structural damage or abnormal behavior.

Earlier stages of the project established baseline static-response models and dynamic vibration-response models using MATLAB-based processing and synchronized sensor measurements. However, real sensor systems are affected by environmental conditions, especially temperature variation.

If temperature drift is not accounted for, the digital twin may incorrectly classify normal thermal sensor behavior as a structural anomaly.

The objective of this stage is therefore to:

- Incorporate temperature as a conditioning variable
- Adjust expected-response bounds accordingly
- Evaluate the effectiveness of temperature compensation
- Reduce false behavioral alarms caused by thermal drift

2 Background

Load cells, strain sensors, amplifiers, and other measurement hardware commonly experience thermal drift during operation. Even when the physical structure remains unchanged, sensor output may slowly vary as temperature changes.

For a behavioral digital twin, this creates an important problem.

Without compensation, the digital twin may interpret temperature-induced drift as:

- Structural deformation
- Sensor instability
- Abnormal loading behavior
- Cyber-physical anomaly

The result is an increase in false alarms.

The purpose of environmental compensation is therefore to separate true structural behavior from predictable environmental effects.

3 Implementation Overview

The environmental compensation workflow is implemented in the MATLAB file:

`temperature_compensation_model.m`

The current implementation uses generated dummy data to simulate realistic temperature drift behavior. This approach was used because full temperature-response experimental data had not yet been collected at the time of development.

However, the implementation was intentionally designed so that the dummy dataset can later be replaced directly with experimental measurements imported from an Excel or CSV file.

The workflow can therefore transition easily from simulated baseline development into real experimental deployment.

4 Dummy Dataset Generation

The MATLAB implementation generates a representative dataset containing:

- Applied load variations
- Slowly varying temperature drift
- Sensor noise
- Temperature-dependent measurement bias

The temperature profile was intentionally designed to mimic realistic laboratory heating behavior during extended operation.

The applied load profile contains several step changes so that the model experiences multiple operating conditions throughout the simulated experiment.

5 Temperature-Dependent Sensor Drift Model

The simulated measured response is modeled as:

$$y_{\text{measured}} = ax + b + c(T - T_{\text{ref}}) + n$$

where:

- x is the applied load
- a is the structural load coefficient
- b is the baseline offset
- c is the temperature drift coefficient
- T is the measured temperature
- T_{ref} is the reference calibration temperature
- n represents random measurement noise

The term:

$$c(T - T_{\text{ref}})$$

represents temperature-induced sensor drift.
This relationship is implemented directly in:

`temperature_compensation_model.m`

during generation of the dummy measured-load signal.

6 Static Twin Without Compensation

The first model implemented in MATLAB ignores temperature entirely.

This uncorrected model assumes the measured response depends only on applied load:

$$\hat{y}_{\text{static}} = ax + b$$

This relationship is generated using:

```
polyfit(AppliedLoad,MeasuredLoad,1)
```

The resulting residual is:

$$r_{\text{static}} = y_{\text{measured}} - y_{\text{expected}}$$

Since the static model ignores thermal drift, the residual grows as temperature changes throughout the experiment.

This causes the digital twin to incorrectly interpret normal environmental drift as abnormal behavior.

7 Temperature-Aware Twin Model

The second MATLAB implementation incorporates temperature directly into the expected-response model.

The temperature-aware regression model is written as:

$$y_{\text{expected}} = b_0 + b_1 L + b_2 (T - T_{\text{ref}})$$

where:

- L is applied load
- T is measured temperature
- b_2 is the learned temperature-drift coefficient

This model is implemented in MATLAB using:

```
beta = X \ MeasuredLoad;
```

where the regression matrix contains both load and temperature terms.

The learned temperature coefficient allows the digital twin to separate thermal drift from actual structural response.

8 Temperature-Corrected Measurement

After estimating the thermal drift coefficient, the MATLAB implementation computes a corrected load signal:

$$y_{\text{corrected}} = y_{\text{measured}} - b_2 (T - T_{\text{ref}})$$

This operation removes the predicted thermal contribution from the measured response.

The corrected signal more closely matches the true applied loading behavior.

9 Residual-Based Compensation Analysis

The primary goal of compensation is to reduce false residual growth.

The residual without compensation is:

$$r_{\text{static}} = y_{\text{measured}} - y_{\text{static}}$$

The compensated residual is:

$$r_{\text{comp}} = y_{\text{measured}} - y_{\text{temp-aware}}$$

The standard deviation of the residual is used to define the acceptable operating envelope:

$$\text{tol} = 2\sigma$$

If the uncompensated residual grows because of temperature drift, the digital twin may generate unnecessary anomaly flags.

Compensation reduces this effect by removing predictable environmental behavior from the residual signal.

10 Environmental Compensation Results

Figure 1 shows the complete temperature-compensation workflow.

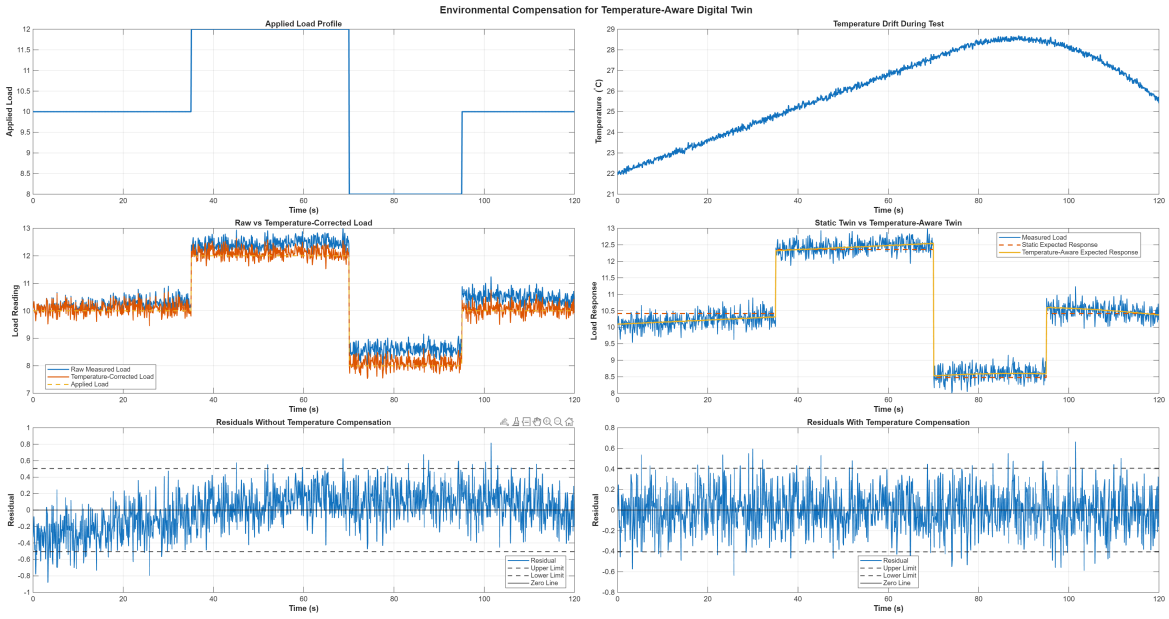


Figure 1: Temperature-aware digital twin compensation workflow and residual comparison.

The results show several important behaviors:

- Temperature increases gradually during the test
- The raw measured load drifts upward with temperature
- The uncompensated static model produces growing residual error
- The temperature-aware model tracks the measured response more accurately
- Residual magnitude decreases significantly after compensation

The lower residual magnitude indicates that the digital twin is correctly separating environmental drift from actual structural behavior.

11 Residual Distribution Comparison

Figure 2 compares the residual distributions before and after compensation.

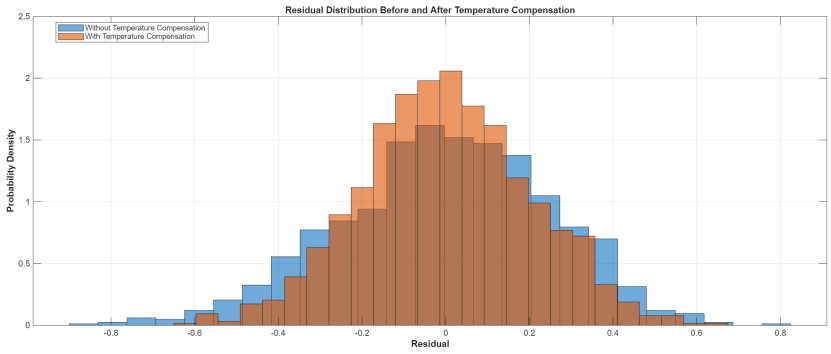


Figure 2: Residual distribution before and after temperature compensation.

The compensated residual distribution is narrower and more centered around zero.

This indicates that the environmental compensation model successfully reduces systematic drift behavior.

A narrower residual distribution is important because it reduces false behavioral alarms within the digital twin.

12 False Alarm Reduction

One of the main performance goals of environmental compensation is reduction of false anomaly flags.

Without compensation, the residual grows primarily because of environmental drift rather than actual structural change.

The MATLAB implementation therefore compares:

- False alarm rate before compensation
- False alarm rate after compensation

The results show that temperature-aware compensation reduces residual spread and improves behavioral consistency.

This is especially important for later AI-based monitoring systems, since poorly compensated residuals may cause unnecessary anomaly detection events.

13 Digital Twin Relevance

Environmental compensation is an important requirement for long-duration digital twin operation.

Without compensation, normal environmental variation may dominate the residual signal and reduce confidence in the monitoring system.

The temperature-aware model improves the digital twin by:

- Reducing environmentally induced residual drift
- Improving expected-response accuracy
- Reducing false alarms
- Improving long-term monitoring stability
- Producing cleaner AI-consumable deviation metrics

The environmental compensation framework also integrates naturally with the larger telemetry and AI-monitoring architecture being developed by the CpE teams.

14 Future Experimental Integration

Although the current implementation uses generated dummy data, the MATLAB workflow was intentionally designed to support future experimental integration with minimal modification.

The generated dataset can later be replaced directly by importing:

- Excel files

- CSV datasets
- Live telemetry streams

Once experimental temperature-response measurements are available, the same compensation pipeline can be applied directly to real structural data.

This allows the current implementation to function both as:

- A proof-of-concept compensation framework
- A deployable future processing architecture

15 Summary

Week 7 introduced environmental compensation into the behavioral digital twin framework.

The MATLAB implementation demonstrated how temperature drift can produce false residual growth and unnecessary anomaly behavior if environmental effects are ignored.

By incorporating temperature as a conditioning variable, the temperature-aware model successfully reduced residual spread, improved expected-response accuracy, and reduced false alarms.

Although the current implementation uses generated dummy data, the workflow was intentionally designed for straightforward replacement with future experimental datasets and live telemetry measurements.

A Appendix A: MATLAB Implementation Overview

The environmental compensation workflow described throughout this report is implemented in the MATLAB file:

`temperature_compensation_model.m`

The implementation generates representative thermal-drift behavior, computes temperature-aware compensation models, evaluates residual behavior before and after compensation, and produces the visualization figures presented throughout this report.

Although the current implementation uses generated dummy data, the workflow was intentionally designed so that experimental Excel or CSV datasets can later be imported directly with minimal modification to the processing pipeline.

The associated MATLAB implementation and generated figures have been uploaded to the project GitHub repository for reproducibility and future experimental integration.

The excerpt below shows the core compensation logic used to estimate and remove temperature-dependent sensor drift.

```
%% Generate measured load response with temperature drift

MeasuredLoad = trueSlope*AppliedLoad ...
             + trueIntercept ...
             + tempCoeff*(Temp - Tref) ...
             + noise;

%% Uncorrected static twin model

pStatic = polyfit(AppliedLoad, MeasuredLoad, 1);

Expected_Static = polyval(pStatic, AppliedLoad);

Residual_Static = MeasuredLoad - Expected_Static;

%% Temperature-aware twin model

X = [ones(N,1), AppliedLoad, Temp - Tref];

beta = X \ MeasuredLoad;
```

```
b0 = beta(1);
bLoad = beta(2);
bTemp = beta(3);

Expected_TempAware = X * beta;

Residual_TempAware = MeasuredLoad - Expected_TempAware;

%% Temperature-corrected measurement

MeasuredLoad_Corrected = ...
    MeasuredLoad - bTemp*(Temp - Tref);

%% Residual statistics

sigmaStatic = std(Residual_Static);

sigmaTempAware = std(Residual_TempAware);

tolStatic = 2 * sigmaStatic;

tolTempAware = 2 * sigmaTempAware;
```

The compensation workflow estimates the thermal-drift coefficient directly from the measured response and removes the predicted thermal contribution from the signal. This reduces environmentally induced residual growth and improves behavioral consistency within the digital twin framework.